

程式碼實驗室：使用 Gemini 以 JavaScript 建構 Chrome 擴充功能

來源：<https://codelabs.developers.google.com/create-chrome-extension-duetai?hl=zh-tw>

步驟數：7

在這個程式碼研究室中，我們會使用 Gemini 建立 Chrome 擴充功能。我們會不斷加入疊代功能，以便為 Google Meet 頁面新增功能。

目錄

1. [1. 簡介](#)
2. [2. 事前準備](#)
3. [3. 讓我們一同展開歡樂旅程吧](#)
4. [4. 新增內容指令碼](#)
5. [5. 傳送通知給使用者](#)
6. [6. 在 Chrome 擴充功能中新增訊息傳遞功能](#)
7. [7. 恭喜](#)

步驟 1 · 約 0 分鐘

1. 簡介

你要加入 Meet 通話，但不想成為第一個加入的人嗎？如果符合上述情況，我們有解決方案可以幫你！

完成本程式碼研究室後，您將建立 Chrome 擴充功能，在第一位出席者加入通話時收到提醒。

您將瞭解 Chrome 擴充功能的不同元素，然後深入瞭解擴充功能的每個部分。您將瞭解內容指令碼、服務工作人員和訊息傳遞等擴充功能。

您必須遵守 [Manifest V3 版本](#)，才能在參與者加入 Meet 通話時收到通知。

2. 事前準備

必要條件

本程式碼研究室適合初學者，但具備 JavaScript 基礎知識可大幅提升學習體驗。

設定/需求

- Chrome 瀏覽器
- 在本機系統上設定 IDE/編輯器。
- 如要使用 gcloud 啟用 Gemini API，請安裝 [gcloud CLI](#)。

啟用 Gemini API

- 在 [Google Cloud 控制台](#) 的專案選取器頁面中，選取或建立 Google Cloud [專案](#)。
- 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。
- 前往 [Gemini Marketplace 頁面](#) 啟用 API。您也可以使用 gcloud 啟用 API：`gcloud services enable clouddaicompanion.googleapis.com --project PROJECT_ID`
- 在新分頁中前往 [Gemini for Cloud 控制台頁面](#)，然後點按「開始對話」。

Note that if you're writing the code in the Cloud Shell editor, then you will have to download the folder somewhere on your local filesystem to test the extension locally.

3. 讓我們一同展開歡樂旅程吧

有時 Gemini 對話回覆可能過於具體，或您希望特定回覆能以全新狀態生成。這時請使用「重設對話」/「清除對話」功能。

基本擴充功能安裝

我們來建立一個目錄，做為專案的根目錄。

〇〇〇

```
mkdir gemini-chrome-ext
cd gemini-chrome-ext
```

在開始詢問 Gemini 特定問題之前，我們先問一些關於 Chrome 擴充功能一般結構的問題。

提示：

〇〇

What are the important parts to build a chrome extension?

我們會收到回應，其中會指定 `manifest` 檔案的次要詳細資料、`background script`，以及使用者介面的詳細資料。讓我們進一步瞭解這些特定檔案。

提示：

```
Create a manifest.json file to build a chrome extension.
Make the name of the extension "Meet Joinees Notifier"
and the author "<YOUR_EMAIL>"
```

您可以在作者欄位中使用所需名稱和電子郵件地址。

Gemini 會傳回我們需要的資訊清單檔案內容，但也會傳回一些我們不需要的額外欄位，例如 `action` 欄位。此外，我們需要說明。讓我們一起解決這個問題！

提示：

```
Remove the "action" field and make the description as
"Adds the ability to receive a notification when a participant joins a Google meet".
```

請將這項內容放入專案根目錄的 `manifest.json` 檔案。

此時，資訊清單檔案應如下所示。

```
{
  "name": "Meet Joinees Notifier",
  "version": "1.0",
  "manifest_version": 3,
  "description": "Adds the ability to receive a notification when a participant joins a
Google Meet",
  "author": "<YOUR_EMAIL>"
}
```

目前請移除資訊清單檔案中產生的任何其他額外欄位，因為本程式碼研究室會假設資訊清單檔案中有這些欄位。

回覆中可能會出現更多欄位。您完全可以嘗試使用含有這些欄位的資訊清單檔案進行實驗。但本程式碼研究室假設您已具備上述內容。

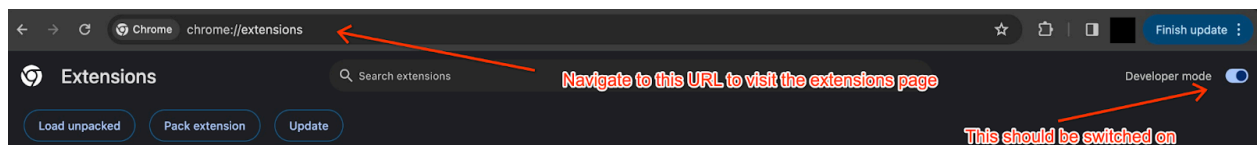
現在，我們要如何測試擴充功能是否正常運作？我們來問問 Gemini 吧！

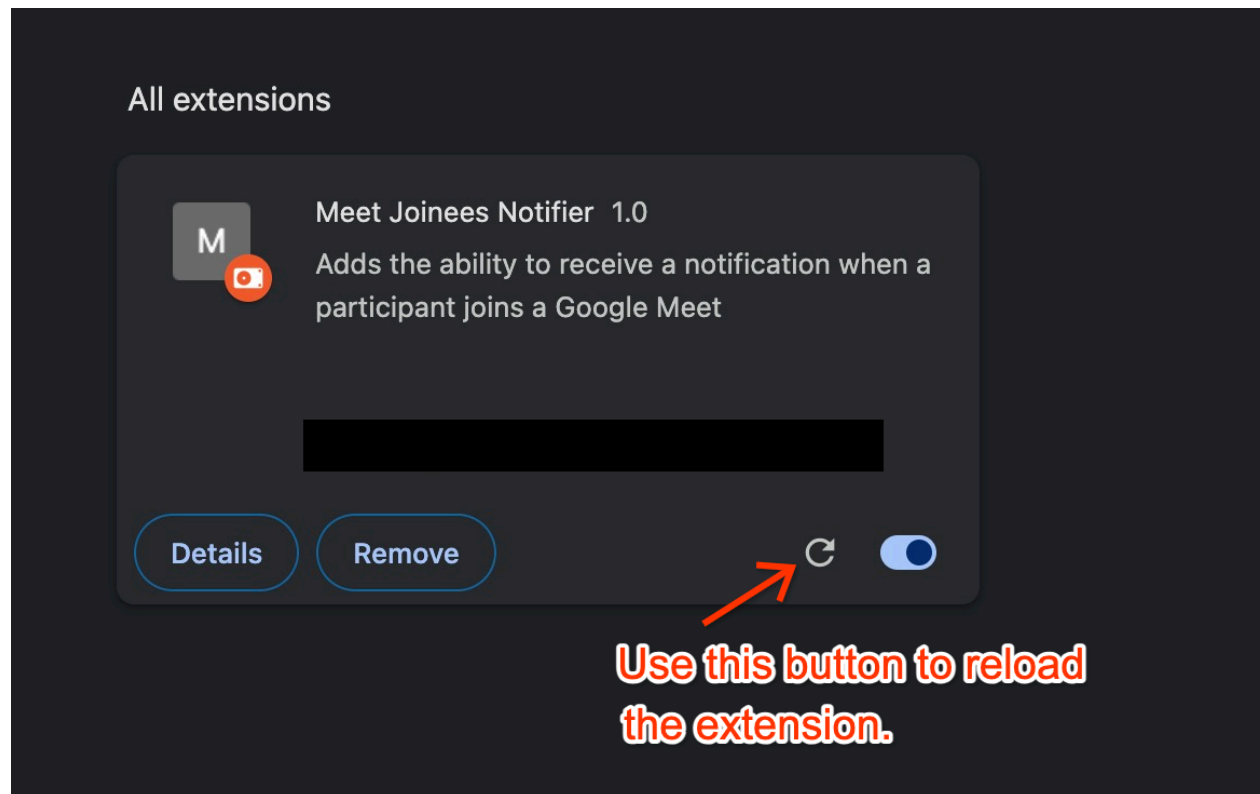
提示：

Guide me on the steps needed to test a chrome extension on my local filesystem.

但有提供測試步驟。請前往 `chrome://extensions` 導覽至 "Extensions Page"，並務必啟用 "Developer Mode" 按鈕，這時應該會顯示 "Load unpacked" 按鈕，可用於導覽至含有擴充功能檔案的本機資料夾。完成後，我們應該就能在 "Extensions Page" 中看到擴充功能。

因為前往擴充功能頁面的方式有很多種，而且不同 Chrome 版本的操作方式也不一樣，因此 Gemini 可能會提供不同的回覆。撰寫本文時，前往 `chrome://extensions` 是最簡單的方法。





太好了！我們可以看到擴充功能，但現在要開始新增一些功能。

步驟 4 · 約 0 分鐘

4. 新增內容指令碼

我們想只在 `https://meet.google.com` 上執行一些 JavaScript 程式碼，這可以使用內容指令碼完成。我們來問問 Gemini，如何在擴充功能中達成這個目標。

提示：

```
How to add a content script in our chrome extension?
```

或更具體地說：

提示：

```
How to add a content script to run on meet.google.com subdomain in our chrome extension?
```

或另一個版本：

提示：

```
Help me add a content script named content.js to run on meet.google.com subdomain in our chrome extension. The content script should simply log "Hello Gemini" when we navigate to "meet.google.com".
```

Gemini 會提供我們需要在 `manifest.json` 檔案中進行的確切變更，以及 `content.js` 檔案中需要的 JavaScript。

加入 `content_scripts` 後，資訊清單檔案會變成：

```
{
  "name": "Meet Joinees Notifier",
  "version": "1.0",
  "manifest_version": 3,
  "description": "Adds the ability to receive a notification when a participant joins a Google Meet",
  "author": "abc@example.com",
  "content_scripts": [
    {
      "matches": ["https://meet.google.com/*"],
      "js": ["content.js"]
    }
  ]
}
```

這會告知 Chrome，每當我們前往「<https://meet.google.com>」子網域中的網頁時，都要插入內容指令碼 `content.js`。我們來新增這個檔案並測試看看。

請注意，每次變更擴充功能的結構(例如資訊清單檔案中的變更)，都必須重新載入擴充功能。或者，你也可以直接詢問 Gemini 該怎麼做 😊

我們在 `content.js` 檔案中新增這段程式碼。

```
console.log("Hello Gemini");
```

果然！前往 meet.google.com 時，我們會在 JavaScript 控制台(Mac：`Cmd + Opt + J` / Windows/Linux：`Ctrl + Shift + J`)中看到「Hello Gemini」。

請注意，我們不會「加入」會議，因為我們會專注於從可加入會議的頁面接收信號。

manifest.json

```
{
```

```
"name": "Meet Joinees Notifier",

"version": "1.0",

"manifest_version": 3,

"description": "Adds the ability to receive a notification when a participant joins a Google Meet",

"author": "luke@cloudadvocacyorg.joonix.net",

"permissions": [

    "tabs",

    "notifications"

],

"content_scripts": [

    {

        "matches": [

            "https://meet.google.com/*"

        ],

        "js": [

            "content.js"

        ]

    }

]
```

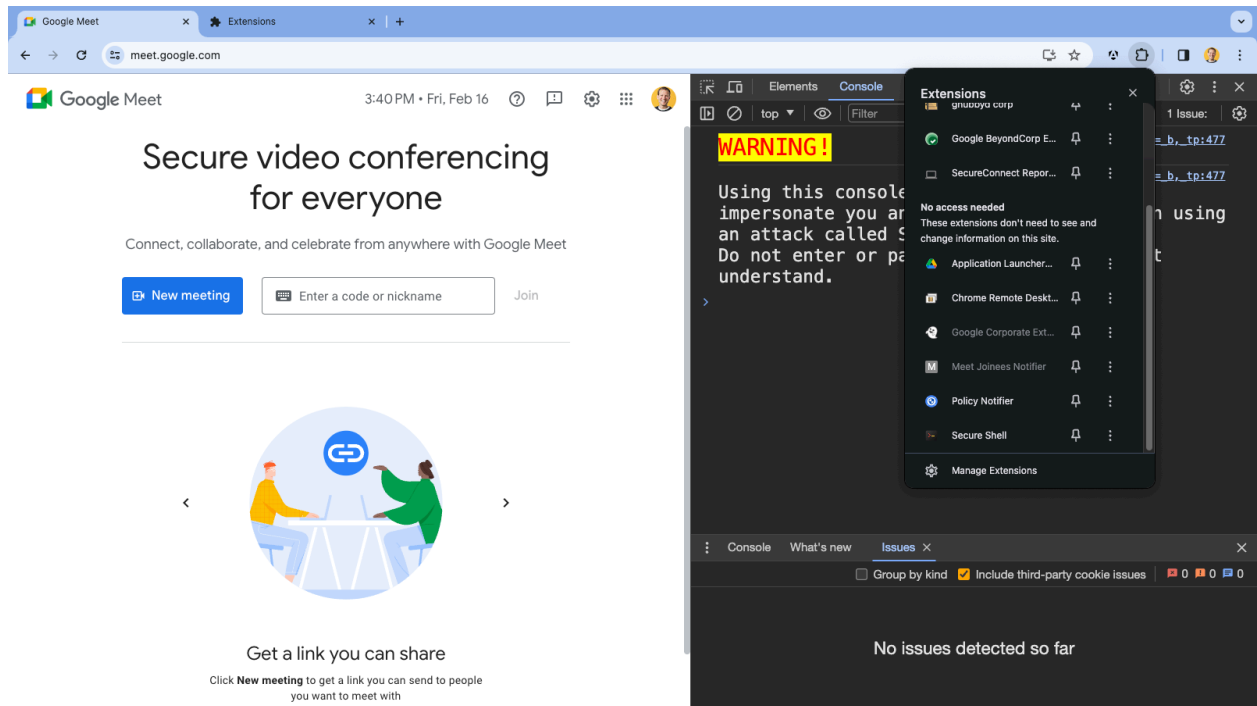
```
}
```

```
□
```

content.js

```
□console.log("Hello Gemini!");
```

```
□
```

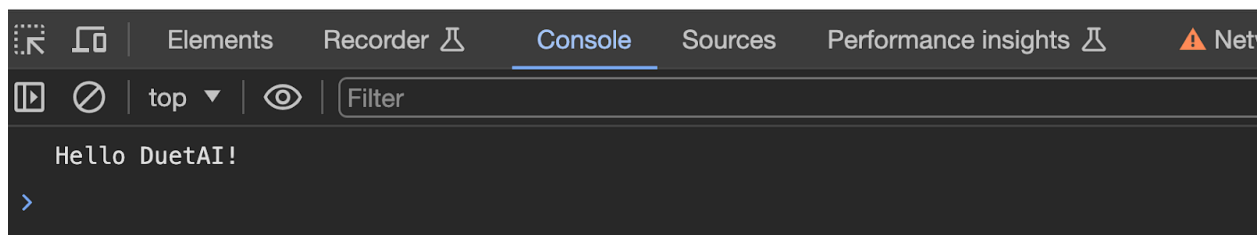


Secure video conferencing for everyone

Connect, collaborate, and celebrate from anywhere with Google Meet

[New meeting](#) [Join](#)

[Learn more](#) about Google Meet

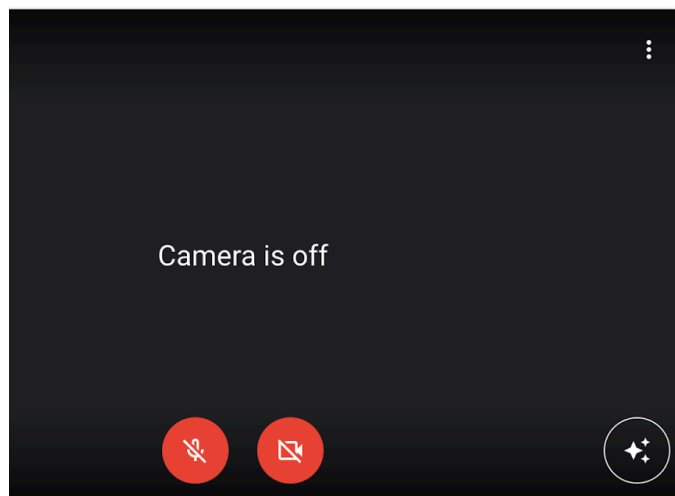


太好了！現在我們已準備好為應用程式新增一些 JavaScript 專屬功能。請花點時間思考我們想達成什麼目標。

改善內容指令碼

我們希望在會議頁面(可選擇加入會議)時，有人加入會議時能收到通知。為達成這個目標，請觀察會議沒有人加入時的畫面，以及有人加入會議時的畫面，看看兩者有何不同。

如果會議中沒有人，畫面會顯示如下。



Ready to join?

No one else is here

Join now

Present

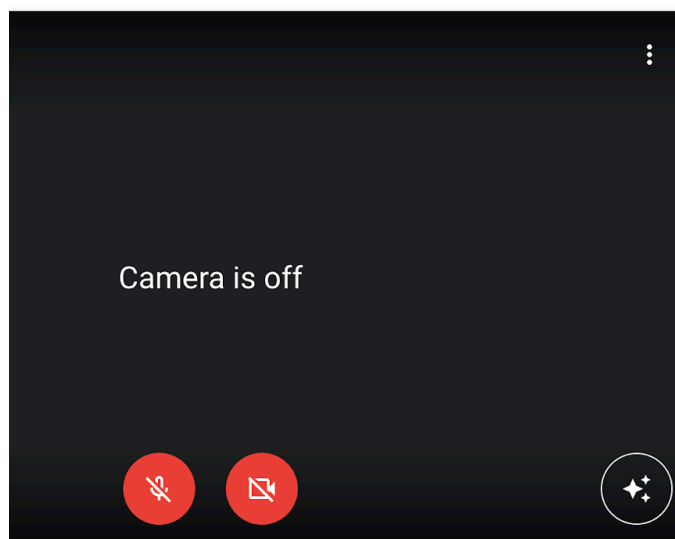
Other joining options

Use Companion mode

Join and use a phone for audio

Connect a conference room

如果會議中有參與者，畫面會如下所示。



Ready to join?



Saransh Dhingra is in this call

Join now

Present

Other joining options

Use Companion mode

Join and use a phone for audio

Connect a conference room

我們馬上就能看出 2 項顯著差異：

1. 狀態文字會從「沒有其他人在這裡」變更為「[使用者] 正在通話中」。
2. 我們可以查看加入通話的使用者圖片。

如果我們想知道是否有人加入會議，這兩項變更都適用，但後者有機會取得已加入使用者的資訊，因此我們嘗試使用後者。

使用快速鍵 (Mac: `Cmd + Opt + C` / Win: `Ctrl + Shift + C`) 在 Chrome 中開啟「元素檢查器」，然後按一下已加入的使用者圖片。

我們可以發現圖片含有幾個類別字串，且圖片的 `title` 屬性包含加入會議的使用者名稱。此外，這個圖片標記會包在類別為 `U04fid` 的 `div` 中。將一些參與者加入測試會議後，我們可以看到這個父項 `div` 會代管多張圖片(對應不同使用者)。

因此，我們已擬定部分策略：

1. 偵測類別為 `U04fid` 的 `div` 是否包含任何元素。
2. 如果有的話，表示會議中有加入者。

3. 通知使用者。

回到 Gemini，詢問如何逐步完成這項作業。

提示：

```
How to detect for element changes in the HTML page?
```

雖然這項要求會提供涉及 `MutationObserver` 的解決方案，但與我們的需求無關，因此請嘗試修改提示。

提示：

```
How can I detect if an element "div.U04fid" has any child images in the HTML page?
```

這次我們取得的解決方案不包含 `MutationObserver`，但會使用父項 `div` 檢查加入者。

我們收到的內容如下：

```
const divU04fid = document.querySelector('div.U04fid');
const childImages = divU04fid.querySelectorAll('img');
if (childImages.length > 0) {
  // div.U04fid has at least one child image.
}
```

讓我們進一步修改做法：

提示：

```
Create a method which checks if the element div.U04fid has any child elements,
if it does it should log it on the console. The method should be called checkJoinees
and it should be called as a callback for MutationObserver which is observing the
document.body.
```

並傳回下列程式碼片段：

```
function checkJoinees(mutations) {
  for (const mutation of mutations) {
    if (mutation.type === 'childList') {
      // A child node was added or removed.
      if (document.querySelector('div.U04fid') &&
document.querySelector('div.U04fid').childElementCount > 0) {
        // div.U04fid has at least one child element.
        console.log('div.U04fid has at least one child element.');
```

請注意，由於達成這項結果的方法有很多種，Gemini 可能會針對這類提示詞傳回不同的結果。您可以試用提示詞的輸出內容、調整提示詞，並嘗試不同的結果，但本程式碼研究室會沿用上述程式碼。

將這段程式碼複製到 `content.js` 檔案中，重新載入擴充功能(然後重新載入 Meet 網頁)。

現在有人加入會議時，我們就能在控制台中看到記錄陳述式。

5. 傳送通知給使用者

我們現在可以偵測參與者何時加入會議，接下來請嘗試在 Chrome 擴充功能中新增通知部分。我們可以瀏覽 [Chrome 擴充功能的說明文件](#)，甚至調整提示來瞭解要尋找的內容，但基本上我們需要使用 `chrome.notifications.create` API，且對這個方法的呼叫應來自背景 Service Worker。

提示：

```
Using the documentation for chrome notifications tell me how to use the
chrome.notifications.create method.
```

我們看到一些詳細步驟，重點如下：

- 在資訊清單檔案中新增 `notifications` 權限。
- 呼叫 `chrome.notifications.create`
- 通話應位於背景指令碼中。

如要在 `manifest version 3` 中將背景指令碼新增至 Chrome 擴充功能，我們需要在 `manifest.json` 檔案中加入 `background.service_worker` 宣告。

請注意，我們使用「Manifest V3」關鍵字，取得有關 Chrome 擴充功能的最新回覆。省略這個屬性仍可取得結果，但有時可能會參照資訊清單 V2。

因此，我們建立名為 `background.js` 的檔案，並在 `manifest.json` 檔案中加入下列內容。

```
"background": {
  "service_worker": "background.js"
},
"permissions": [
  "notifications"
]
```

加入上述內容後，資訊清單檔案會變成：

```
{
  "name": "Meet Joinees Notifier",
  "version": "1.0",
  "manifest_version": 3,
  "description": "Adds the ability to receive a notification when a participant joins a
Google Meet",
  "author": "<YOUR_EMAIL>",
  "content_scripts": [
    {
      "matches": ["https://meet.google.com/*"],
      "js": ["content.js"]
    }
  ],
  "background": {
    "service_worker": "background.js"
  },
  "permissions": [
    "notifications"
  ]
}
```

提示：

Create a method `sendNotification` that calls the `chrome.notifications.create` method with the message, "A user joined the call" for a chrome extension with manifest v3, the code is in the background service worker

將這張圖片儲存在資料夾的根目錄，並重新命名為 `success.png`。



接著，將下列程式碼片段新增至 `background.js`。

```
function sendNotification(notificationId, message) {
  chrome.notifications.create(notificationId, {
    type: "basic",
    title: "A user joined the call",
    message: message,
    imageUrl: "./success.png"
  });
}

sendNotification("notif-id", "test message");
```

現在從擴充功能頁面重新載入擴充功能，您應該會立即看到通知彈出式視窗。

如果沒有看到通知，請確認 Chrome 瀏覽器是否具備顯示通知的必要權限 ([Windows](#)、[Mac](#) 的操作說明)

6. 在 Chrome 擴充功能中新增訊息傳遞功能

現在，我們需要執行的最後一個主要步驟，是連結內容指令碼對參與者的偵測，以及背景指令碼中的 `sendNotification` 方法。在 Chrome 擴充功能的脈絡中，方法是透過稱為「`message passing`」的技術。

這項功能可讓 Chrome 擴充功能的不同部分進行通訊，在本例中，是從內容指令碼到背景 Service Worker。我們來詢問 Gemini 該怎麼做。

提示：

```
How to send a message from the content script to the background script in a chrome extension
```

Gemini 會回覆對 `chrome.runtime.sendMessage` 和 `chrome.runtime.onMessage.addListener` 的相關呼叫。

基本上，我們會使用 `sendMessage` 從內容指令碼傳送訊息，指出有人已加入 Meet 通話，並使用 `onMessage.addListener` 做為事件監聽器，對內容指令碼傳送的訊息做出反應。在本例中，我們會從這個事件監聽器觸發對 `sendNotification` 方法的呼叫。

我們會將通知訊息和 `action` 屬性傳遞至背景 Service Worker。`action` 屬性說明背景指令碼回應的內容。

因此，我們的 `content.js` 程式碼如下：

```
function checkJoinees(mutations) {
  for (const mutation of mutations) {
    if (mutation.type === 'childList') {
      // A child node was added or removed.
      if (document.querySelector('div.U04fid') &&
document.querySelector('div.U04fid').childElementCount > 0) {
        // div.U04fid has at least one child element.
        sendMessage();
      }
    }
  }
  return false;
}

const observer = new MutationObserver(checkJoinees);
observer.observe(document.body, {
  childList: true,
  delay: 1000
});

function sendMessage() {
  chrome.runtime.sendMessage({
    txt: "A user has joined the call!",
    action: "people_joined"
  });
}
```

這是我們的 `background.js` 程式碼：

```
chrome.runtime.onMessage.addListener((message, sender, sendResponse) => {
  if (message.action === "people_joined") {
    sendNotification("notif-id", message.txt);
  }
});

function sendNotification(notificationId, message) {
  chrome.notifications.create(notificationId, {
    type: "basic",
    title: "A user joined the call",
    message: message,
    imageUrl: "./success.png"
  });
}
```

讓我們試著自訂通知訊息，並取得專屬通知 ID。通知訊息可以包含使用者名稱。如果回想一下先前的步驟，我們可以看到圖片的 title 屬性中顯示使用者名稱。因此，我們可以使用 `document.querySelector('div.U04fid > img').getAttribute('title')` 擷取參與者的名稱

至於通知 ID，我們可以擷取內容指令碼的分頁 ID，並將其做為通知 ID。在事件監聽器

`chrome.runtime.onMessage.addListener` 中，使用 `sender.tab.id` 即可完成這項操作

使用分頁 ID 表示每個分頁只會有一份通知副本。邀請您思考如何改變這種行為，以達到下列目標：

1. 每次通話都會收到專屬通知
2. 不同會議 ID 的專屬通知。

最後，我們的檔案應如下所示：

manifest.json

```
{
  "name": "Meet Joinees Notifier",
  "version": "1.0",
  "manifest_version": 3,
  "description": "Adds the ability to receive a notification when a participant joins a
Google Meet",
  "author": "<YOUR_EMAIL>",
  "content_scripts": [
    {
      "matches": ["https://meet.google.com/*"],
      "js": ["content.js"]
    }
  ],
  "background": {
    "service_worker": "background.js"
  },
  "permissions": [
    "notifications"
  ]
}
```

content.js

```

function checkJoinees(mutations) {
  for (const mutation of mutations) {
    if (mutation.type === 'childList') {
      // A child node was added or removed.
      if (document.querySelector('div.U04fid') &&
document.querySelector('div.U04fid').childElementCount > 0) {
        const name = document.querySelector('div.U04fid > img').getAttribute('title');
        sendMessage(name);
      }
    }
  }
  return false;
}

const observer = new MutationObserver(checkJoinees);
observer.observe(document.body, {
  childList: true,
  delay: 1000
});

function sendMessage(name) {
  const joinee = (name === null ? 'Someone' : name),
    txt = `${joinee} has joined the call!`;

  chrome.runtime.sendMessage({
    txt,
    action: "people_joined",
  });
}

```

background.js

```

chrome.runtime.onMessage.addListener((message, sender, sendResponse) => {
  if (message.action === "people_joined") {
    sendNotification("" + sender.tab.id, message.txt); // We are casting this to string as
notificationId is expected to be a string while sender.tab.id is an integer.
  }
});

function sendNotification(notificationId, message) {
  chrome.notifications.create(notificationId, {
    type: "basic",
    title: "A user joined the call",
    message: message,
    imageUrl: "./success.png"
  });
}

```

7. 恭喜

在 Gemini 的協助下，我們很快就建構出 Chrome 擴充功能。無論您是經驗豐富的 Chrome 擴充功能開發人員，還是擴充功能新手，Gemini 都能協助您完成想達成的任何工作。

建議你詢問 Chrome 擴充功能可執行的各種操作。有許多值得瀏覽的 API，例如 `chrome.storage`、`alarms` 等。如果遇到困難，請使用 Gemini 或說明文件瞭解錯誤之處，或收集解決問題的不同方法。

通常需要修改提示才能取得所需協助，但我們可以在同一個分頁中執行這項操作，並保留所有脈絡歷程。
